



BITS Pilani
Compiler Construction (CS F363)
Phase 2 Report: Parsing and Intermediate Code
Generation
Minimal Modula-2 Subset

Group No. 13

Arav Thakkar - 2023A7PS1091H
Mokshil Shah - 2023A7PS0139H
Harshal Shah - 2023A7PS0055H
Marmik Sapovadia - 2023A7PS0057H
Akshat Kumar - 2023A7PS0117H

Contents

1	Introduction	3
2	System Architecture	3
3	Lexer Implementation	3
3.1	Key Features	3
4	Parser Implementation	3
4.1	Features	3
4.2	Semantic Value Structure	4
5	Three Address Code (TAC) Generation	4
5.1	Temporary Variables	4
5.2	Label Generation	4
6	TAC for Expressions	4
7	TAC for Conditional Statements	4
8	TAC for While Loop	5
9	Testing	5
9.1	Valid Input	5
9.2	Invalid Input	5
10	Challenges Faced	6
11	Conclusion	6

1 Introduction

In this phase, the compiler for a minimal subset of the Modula-2 language was extended to include:

- Syntax analysis using a Bison-based parser
- Intermediate code generation using Three Address Code (TAC)

The system integrates the following pipeline:

Input Program	Lexer	Parser	TAC Generator	Output
---------------	-------	--------	---------------	--------

2 System Architecture

Input (.mod) → Lexer (Flex) → Parser (Bison) → TAC Output

3 Lexer Implementation

The lexical analyzer was implemented using Flex.

3.1 Key Features

- Recognizes keywords such as MODULE, BEGIN, END, IF, WHILE
- Supports operators like :=, i=, i=, i=
- Identifies identifiers and numbers
- Stores lexeme values using `yylval.sval`
- Handles comments using exclusive states
- Reports lexical errors

4 Parser Implementation

The parser was implemented using Bison.

4.1 Features

- Left-recursive grammar retained
- Semantic values handled using `%union`
- Error handling using `yyerror()`

4.2 Semantic Value Structure

```
%union {  
    char *sval;  
}
```

5 Three Address Code (TAC) Generation

TAC is generated during parsing.

5.1 Temporary Variables

```
t1, t2, t3, ...
```

5.2 Label Generation

```
L1, L2, L3, ...
```

6 TAC for Expressions

```
x := a + b * c
```

```
t1 = b * c  
t2 = a + t1  
x = t2
```

7 TAC for Conditional Statements

```
IF cond THEN S1 ELSE S2 END
```

```
if cond goto L1  
goto L2  
L1:  
...  
goto L3  
L2:  
...  
L3:
```

8 TAC for While Loop

```
WHILE cond DO S END
```

```
L1:  
if cond goto L2  
goto L3  
L2:  
...  
goto L1  
L3:
```

9 Testing

9.1 Valid Input

```
MODULE Test;  
BEGIN  
  VAR x : INTEGER;  
  VAR y : INTEGER;  
  
  x := 5;  
  y := x + 3 * 2;  
  
  IF y > 10 THEN  
    x := y + 1;  
  ELSE  
    x := y - 1;  
  END;  
  
  WHILE x < 20 DO  
    x := x + 2;  
  END;  
  
END Test.
```

9.2 Invalid Input

```
MODULE Test;  
BEGIN  
  VAR x : INTEGER;  
  
  x := 5 + ;  
  
END Test.
```

10 Challenges Faced

- Reduce/reduce conflicts resolved
- Flex library issue on macOS handled
- Duplicate main() error fixed
- Token mismatch issues corrected

11 Conclusion

A complete compiler front-end was implemented, including lexical analysis, parsing, and TAC generation.